📚 Documentación Técnica Completa - Digital Signage Enterprise

🧬 Arquitectura del Sistema

Stack Tecnológico

```yaml
Frontend Admin:
  - Framework: React 18
  - Styling: Tailwind CSS
  - State Management: Zustand
  - Routing: React Router v6
  - HTTP Client: Axios
  - Forms: React Hook Form + Zod

Backend API:
  - Runtime: Node.js 20 LTS
  - Framework: Express.js
  - Language: TypeScript
  - Database: PostgreSQL 15
  - ORM: Prisma
  - Cache: Redis
  - Queue: BullMQ
  - WebSocket: Socket.io
  - Storage: MinIO (S3-compatible)
  - Auth: JWT + Passport

Electron Player:
  - Framework: Electron 28
  - Renderer: React 18
  - IPC: Electron IPC
  - Cache: IndexedDB
  - Auto-update: electron-updater
  - Packaging: electron-builder

DevOps:
  - Container: Docker + Docker Compose
  - CI/CD: GitHub Actions
  - Monitoring: Grafana + Prometheus
  - Logs: Winston + Loki
```

📁 Estructura del Proyecto

```
digital-signage/
├── packages/
│   ├── admin/                  # Admin Panel (React)
│   │   ├── src/
│   │   │   ├── components/     # Componentes reutilizables
│   │   │   ├── views/          # Vistas/páginas
│   │   │   ├── hooks/          # Custom hooks
│   │   │   ├── services/       # API clients
│   │   │   ├── store/          # Zustand stores
│   │   │   ├── types/          # TypeScript types
│   │   │   └── utils/          # Utilidades
│   │   └── package.json
│   │
│   ├── player/                 # Electron Player
│   │   ├── electron/
│   │   │   ├── main/           # Main process
│   │   │   │   ├── index.ts    # Entry point
│   │   │   │   ├── window.ts   # Window manager
│   │   │   │   ├── sync.ts     # Sync service
│   │   │   │   ├── cache.ts    # Cache manager
│   │   │   │   ├── updater.ts  # Auto-updater
│   │   │   │   └── kiosk.ts    # Kiosk mode
│   │   │   ├── preload/
│   │   │   │   └── index.ts    # Preload script
│   │   │   └── renderer/       # UI React
│   │   ├── src/
│   │   └── package.json
│   │
│   └── backend/                # API Server
│       ├── src/
│       │   ├── api/            # API routes
│       │   │   ├── auth/
│       │   │   ├── screens/
│       │   │   ├── media/
│       │   │   ├── ai/
│       │   │   ├── backups/
│       │   │   ├── tickets/
│       │   │   └── analytics/
│       │   ├── services/       # Business logic
│       │   ├── models/         # Database models
│       │   ├── middleware/     # Middlewares
```

```
│   │   ├── utils/          # Utilities
│   │   ├── config/         # Configuration
│   │   └── app.ts          # Express app
│   ├── prisma/
│   │   └── schema.prisma   # Database schema
│   └── package.json
│
├── docker/
│   ├── docker-compose.yml
│   ├── Dockerfile.api
│   └── Dockerfile.admin
│
├── docs/                   # Documentación
├── scripts/                # Scripts de deployment
└── package.json            # Root package
```

🔌 Backend API - Endpoints Completos

1. Authentication

typescript

```ts
/**
 * POST /api/auth/register
 * Registra un nuevo usuario
 */
interface RegisterRequest {
  email: string;
  password: string;
  name: string;
  role: 'admin_it' | 'marketeer';
  organization_id?: string;
}

interface RegisterResponse {
  user: User;
  token: string;
  refreshToken: string;
}

/**
 * POST /api/auth/login
 * Inicia sesión
 */
interface LoginRequest {
  email: string;
  password: string;
}

interface LoginResponse {
  user: User;
  token: string;
  refreshToken: string;
}

/**
 * POST /api/auth/refresh
 * Refresca el token
 */
interface RefreshRequest {
  refreshToken: string;
}

/**
 * POST /api/auth/logout
```

```
 * Cierra sesión
 */

/**
 * GET /api/auth/me
 * Obtiene usuario actual
 */
```

2. Screens

typescript

```typescript
/**
 * GET /api/screens
 * Lista todas las pantallas
 * Query params: ?type=signage&zone=Norte&status=online
 */
interface GetScreensResponse {
  screens: Screen[];
  total: number;
  page: number;
  limit: number;
}

/**
 * GET /api/screens/:id
 * Obtiene una pantalla específica
 */
interface GetScreenResponse {
  screen: Screen;
}

/**
 * POST /api/screens
 * Crea una nueva pantalla
 */
interface CreateScreenRequest {
  name: string;
  type: 'signage' | 'kiosk' | 'dashboard';
  zone: string;
  location: string;
  orientation: 'horizontal' | 'vertical';
  layout: string;
  content: ScreenContent;
  schedule?: Schedule;
}

/**
 * PUT /api/screens/:id
 * Actualiza una pantalla
 */
interface UpdateScreenRequest {
  name?: string;
  zone?: string;
  status?: 'online' | 'offline' | 'maintenance';
```

```
  content?: Partial<ScreenContent>;
  schedule?: Schedule;
}

/**
 * DELETE /api/screens/:id
 * Elimina una pantalla
 */

/**
 * GET /api/screens/:id/preview
 * Obtiene preview de pantalla
 */
interface PreviewResponse {
  html: string;
  assets: string[];
}

/**
 * POST /api/screens/:id/screenshot
 * Solicita screenshot remoto
 */

/**
 * GET /api/screens/:id/logs
 * Obtiene logs de la pantalla
 */
interface GetLogsResponse {
  logs: Log[];
}
```

3. AI Generation

typescript

```typescript
/**
 * POST /api/ai/generate-screen
 * Genera pantalla con Claude AI
 */
interface GenerateScreenRequest {
  prompt: string;
  type?: 'signage' | 'kiosk' | 'dashboard';
  orientation?: 'horizontal' | 'vertical';
}

interface GenerateScreenResponse {
  screen: {
    name: string;
    layout: string;
    zones: Zone[];
    widgets: Widget[];
    theme: Theme;
  };
  reasoning: string;
}

/**
 * POST /api/ai/generate-image
 * Genera imagen con Stability AI
 */
interface GenerateImageRequest {
  prompt: string;
  style?: 'realistic' | 'artistic' | 'minimalist';
  size?: '512x512' | '1024x1024';
}

interface GenerateImageResponse {
  url: string;
  filename: string;
  size: number;
}

/**
 * POST /api/ai/generate-video
 * Genera video con Runway ML
 */
interface GenerateVideoRequest {
  prompt: string;
```

```typescript
  duration?: 5 | 10;
  fps?: 24 | 30;
}

interface GenerateVideoResponse {
  url: string;
  filename: string;
  duration: number;
  size: number;
}

/**
 * POST /api/ai/generate-animation
 * Genera animación CSS
 */
interface GenerateAnimationRequest {
  prompt: string;
  type?: 'fade' | 'slide' | 'zoom' | 'rotate' | 'custom';
}

interface GenerateAnimationResponse {
  css: string;
  json: object;
}
```

4. Media Library

typescript

```
/**
 * GET /api/media
 * Lista archivos multimedia
 * Query params: ?type=video&search=logo
 */
interface GetMediaResponse {
  files: MediaFile[];
  total: number;
}

/**
 * POST /api/media/upload
 * Sube archivo
 * Content-Type: multipart/form-data
 */
interface UploadResponse {
  file: MediaFile;
  url: string;
}

/**
 * DELETE /api/media/:id
 * Elimina archivo
 */

/**
 * GET /api/media/:id/usage
 * Obtiene dónde se usa el archivo
 */
interface MediaUsageResponse {
  screens: Screen[];
  count: number;
}
```

5. Backups

typescript

```typescript
/**
 * GET /api/backups
 * Lista backups
 */
interface GetBackupsResponse {
  backups: Backup[];
}

/**
 * POST /api/backups
 * Crea nuevo backup
 */
interface CreateBackupRequest {
  includeMedia?: boolean;
  compression?: boolean;
}

interface CreateBackupResponse {
  backup: Backup;
  downloadUrl: string;
}

/**
 * GET /api/backups/:id/download
 * Descarga backup
 */

/**
 * POST /api/backups/:id/restore
 * Restaura desde backup
 */
```

6. Tickets

typescript

```typescript
/**
 * GET /api/tickets
 * Lista tickets
 * Query params: ?status=pending&priority=high
 */
interface GetTicketsResponse {
  tickets: Ticket[];
}

/**
 * POST /api/tickets
 * Crea nuevo ticket (desde cajero)
 */
interface CreateTicketRequest {
  cashier: string;
  location: string;
  priority: 'low' | 'medium' | 'high';
  message: string;
}

/**
 * PUT /api/tickets/:id
 * Actualiza estado del ticket
 */
interface UpdateTicketRequest {
  status: 'pending' | 'in_progress' | 'resolved';
}

/**
 * DELETE /api/tickets/:id
 * Elimina ticket
 */

/**
 * WebSocket /api/tickets/subscribe
 * Suscripción a nuevos tickets en tiempo real
 */
```

7. Database Connections

typescript

```typescript
/**
 * GET /api/database/connections
 * Lista conexiones BD
 */
interface GetConnectionsResponse {
  connections: DatabaseConnection[];
}

/**
 * POST /api/database/connections
 * Crea nueva conexión
 */
interface CreateConnectionRequest {
  name: string;
  type: 'postgresql' | 'mysql' | 'mongodb' | 'api';
  config: ConnectionConfig;
}

/**
 * POST /api/database/connections/:id/test
 * Prueba conexión
 */
interface TestConnectionResponse {
  success: boolean;
  latency: number;
  error?: string;
}

/**
 * POST /api/database/query
 * Ejecuta query
 */
interface ExecuteQueryRequest {
  connectionId: string;
  query: string;
}

interface ExecuteQueryResponse {
  data: any[];
  rowCount: number;
  executionTime: number;
}
```

8. Analytics

typescript

```typescript
/**
 * GET /api/analytics/overview
 * Dashboard general
 */
interface AnalyticsOverview {
  totalScreens: number;
  onlineScreens: number;
  totalViews: number;
  averageUptime: number;
  topScreens: Screen[];
}

/**
 * GET /api/analytics/screens/:id
 * Analytics de pantalla específica
 */
interface ScreenAnalytics {
  views: number;
  uptime: number;
  lastSync: Date;
  errors: number;
  performance: {
    avgLoadTime: number;
    avgFps: number;
  };
}

/**
 * GET /api/analytics/content
 * Analytics de contenido
 */
interface ContentAnalytics {
  mostUsedFiles: MediaFile[];
  totalStorage: number;
  byType: Record<string, number>;
}
```

9. API Keys Management

typescript

```typescript
/**
 * GET /api/settings/api-keys
 * Lista API keys configuradas (sin mostrar valores)
 */
interface GetApiKeysResponse {
  keys: {
    service: string;
    configured: boolean;
    lastUsed?: Date;
  }[];
}

/**
 * PUT /api/settings/api-keys/:service
 * Actualiza API key
 */
interface UpdateApiKeyRequest {
  key: string;
  encrypted?: boolean;
}

/**
 * POST /api/settings/api-keys/:service/test
 * Prueba API key
 */
interface TestApiKeyResponse {
  valid: boolean;
  error?: string;
}
```

🔐 Modelos de Base de Datos (Prisma Schema)

prisma

```
// prisma/schema.prisma

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

// Modelo de Usuario
model User {
  id        String   @id @default(cuid())
  email     String   @unique
  password  String   // Hashed con bcrypt
  name      String
  role      Role     @default(MARKETEER)
  avatar    String?

  organizationId String?
  organization   Organization? @relation(fields: [organizationId], references: [id])

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  screens   Screen[]
  backups   Backup[]
  tickets   Ticket[]

  @@index([email])
  @@index([organizationId])
}

enum Role {
  ADMIN_IT
  MARKETEER
}

// Modelo de Organización (Multi-tenancy)
model Organization {
  id          String   @id @default(cuid())
  name        String
```

```
  slug        String   @unique
  plan        Plan     @default(FREE)
  maxScreens  Int      @default(3)

  users       User[]
  screens     Screen[]
  media       MediaFile[]

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([slug])
}

enum Plan {
  FREE
  PRO
  BUSINESS
  ENTERPRISE
}

// Modelo de Pantalla
model Screen {
  id          String   @id @default(cuid())
  name        String
  type        ScreenType
  zone        String
  location    String
  orientation Orientation
  layout      String
  status      ScreenStatus @default(OFFLINE)

  // Contenido
  content     Json     // Almacena configuración completa

  // Programación
  schedule    Json?    // Horarios y reglas

  // Metadata
  lastSync    DateTime?
  lastSeen    DateTime?
  version     String?  // Versión del player
  playerId    String?  // ID único del player
```

```
  aiGenerated Boolean  @default(false)

  userId      String
  user        User     @relation(fields: [userId], references: [id])

  organizationId String
  organization   Organization @relation(fields: [organizationId], references: [id])

  logs        Log[]

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([type])
  @@index([status])
  @@index([organizationId])
  @@index([userId])
}

enum ScreenType {
  SIGNAGE
  KIOSK
  DASHBOARD
}

enum Orientation {
  HORIZONTAL
  VERTICAL
}

enum ScreenStatus {
  ONLINE
  OFFLINE
  SYNCING
  MAINTENANCE
  ERROR
}

// Modelo de Archivo Multimedia
model MediaFile {
  id          String   @id @default(cuid())
  filename    String
  originalName String
  type        MediaType
```

```
  mimeType    String
  size        Int     // Bytes
  url         String
  thumbnailUrl String?

  aiGenerated Boolean  @default(false)
  prompt      String?  // Si fue generado con IA

  usageCount  Int      @default(0)

  organizationId String
  organization   Organization @relation(fields: [organizationId], references: [id])

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([type])
  @@index([organizationId])
}

enum MediaType {
  VIDEO
  IMAGE
  ANIMATION
  AUDIO
  DOCUMENT
}

// Modelo de Backup
model Backup {
  id          String   @id @default(cuid())
  filename    String
  size        Int      // Bytes
  url         String
  status      BackupStatus

  includesMedia Boolean @default(true)
  compressed    Boolean @default(true)

  screensCount  Int
  mediaCount    Int

  userId      String
  user        User     @relation(fields: [userId], references: [id])
```

```
  createdAt   DateTime @default(now())
  completedAt DateTime?

  @@index([userId])
  @@index([status])
}

enum BackupStatus {
  PENDING
  IN_PROGRESS
  COMPLETED
  FAILED
}

// Modelo de Ticket
model Ticket {
  id          String   @id @default(cuid())
  cashier     String
  location    String
  priority    Priority
  message     String
  status      TicketStatus @default(PENDING)

  userId      String?
  user        User?    @relation(fields: [userId], references: [id])

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  resolvedAt  DateTime?

  @@index([status])
  @@index([priority])
  @@index([userId])
}

enum Priority {
  LOW
  MEDIUM
  HIGH
  CRITICAL
}

enum TicketStatus {
```

```
  PENDING
  IN_PROGRESS
  RESOLVED
  CANCELLED
}

// Modelo de Conexión a Base de Datos
model DatabaseConnection {
  id          String   @id @default(cuid())
  name        String
  type        DatabaseType
  host        String
  port        Int
  database    String
  username    String
  password    String   // Encriptado

  ssl         Boolean  @default(false)

  organizationId String

  queries     Query[]

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  lastTested  DateTime?

  @@index([organizationId])
}

enum DatabaseType {
  POSTGRESQL
  MYSQL
  MONGODB
  MSSQL
  ORACLE
  API
}

// Modelo de Query
model Query {
  id          String   @id @default(cuid())
  name        String
  sql         String   @db.Text
```

```
  refreshInterval Int  // Segundos

  connectionId String
  connection   DatabaseConnection @relation(fields: [connectionId], references: [id])

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([connectionId])
}

// Modelo de Log
model Log {
  id         String   @id @default(cuid())
  level      LogLevel
  message    String   @db.Text
  metadata   Json?

  screenId   String?
  screen     Screen?  @relation(fields: [screenId], references: [id])

  createdAt   DateTime @default(now())

  @@index([level])
  @@index([screenId])
  @@index([createdAt])
}

enum LogLevel {
  INFO
  WARNING
  ERROR
  DEBUG
}

// Modelo de API Key
model ApiKey {
  id         String   @id @default(cuid())
  service    ApiService
  key        String   // Encriptado

  organizationId String

  lastUsed   DateTime?
```

```
  usageCount  Int      @default(0)

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@unique([organizationId, service])
  @@index([organizationId])
}

enum ApiService {
  CLAUDE
  OPENAI
  STABILITY
  RUNWAY
  ELEVENLABS
}
```

🎯 Implementación del Backend

1. Configuración de Express

typescript

```typescript
// src/app.ts
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';
import { createServer } from 'http';
import { Server } from 'socket.io';
import { PrismaClient } from '@prisma/client';
import { errorHandler } from './middleware/error.handler';
import { authRoutes } from './api/auth/routes';
import { screenRoutes } from './api/screens/routes';
import { aiRoutes } from './api/ai/routes';
import { mediaRoutes } from './api/media/routes';
import { backupRoutes } from './api/backups/routes';
import { ticketRoutes } from './api/tickets/routes';
import { analyticsRoutes } from './api/analytics/routes';

// Inicializar Prisma
export const prisma = new PrismaClient();

// Crear app Express
const app = express();

// Middleware
app.use(cors({
  origin: process.env.FRONTEND_URL,
  credentials: true
}));
app.use(helmet());
app.use(morgan('combined'));
app.use(express.json({ limit: '50mb' }));
app.use(express.urlencoded({ extended: true, limit: '50mb' }));

// Health check
app.get('/health', (req, res) => {
  res.json({ status: 'ok', timestamp: new Date() });
});

// API Routes
app.use('/api/auth', authRoutes);
app.use('/api/screens', screenRoutes);
app.use('/api/ai', aiRoutes);
app.use('/api/media', mediaRoutes);
```

```
app.use('/api/backups', backupRoutes);
app.use('/api/tickets', ticketRoutes);
app.use('/api/analytics', analyticsRoutes);

// Error handling
app.use(errorHandler);

// Crear servidor HTTP
const httpServer = createServer(app);

// Configurar Socket.io para WebSocket
const io = new Server(httpServer, {
  cors: {
    origin: process.env.FRONTEND_URL,
    credentials: true
  }
});

// Socket.io handlers
io.on('connection', (socket) => {
  console.log('Cliente conectado:', socket.id);

  // Join room por organización
  socket.on('join-organization', (orgId: string) => {
    socket.join(`org-${orgId}`);
  });

  // Ticket updates
  socket.on('ticket-update', (data) => {
    io.to(`org-${data.organizationId}`).emit('ticket-notification', data);
  });

  socket.on('disconnect', () => {
    console.log('Cliente desconectado:', socket.id);
  });
});

export { app, httpServer, io };
```

2. Servicio de Generación con Claude AI

typescript

```
// src/services/ai/claude.service.ts
import Anthropic from '@anthropic-ai/sdk';
import { prisma } from '../../app';

export class ClaudeService {
  private client: Anthropic;

  constructor(apiKey: string) {
    this.client = new Anthropic({ apiKey });
  }

  /**
   * Genera una pantalla usando Claude AI
   */
  async generateScreen(prompt: string, type?: string, orientation?: string) {
    const systemPrompt = `Eres un diseñador experto de digital signage. Tu trabajo es crear configuraciones detalladas para pa

IMPORTANTE: Responde ÚNICAMENTE con JSON válido, sin texto adicional, sin markdown, sin explicaciones.

Formato de respuesta:
{
  "name": "Nombre descriptivo de la pantalla",
  "type": "signage | kiosk | dashboard",
  "orientation": "horizontal | vertical",
  "layout": "nombre del layout (corporate, menu-vertical, analytics, etc)",
  "zones": [
    {
      "id": "zone-1",
      "name": "Header",
      "position": { "x": 0, "y": 0, "width": 100, "height": 15 },
      "type": "header"
    }
  ],
  "widgets": [
    {
      "id": "widget-1",
      "type": "clock | weather | rss | html | chart",
      "zone": "zone-id",
      "config": { }
    }
  ],
  "theme": {
    "primary": "#hex",
```

```
    "secondary": "#hex",
    "background": "#hex",
    "text": "#hex"
  },
  "content": {
    "logo": "emoji o URL",
    "video": "nombre de video",
    "rssText": "texto para banner RSS"
  }
}`;

    const userPrompt = `${prompt}

${type ? `Tipo preferido: ${type}` : ''}
${orientation ? `Orientación preferida: ${orientation}` : ''}

Genera la configuración completa de la pantalla.`;

    try {
      const message = await this.client.messages.create({
        model: 'claude-sonnet-4-20250514',
        max_tokens: 4000,
        temperature: 0.7,
        system: systemPrompt,
        messages: [{
          role: 'user',
          content: userPrompt
        }]
      });

      // Extraer JSON de la respuesta
      const responseText = message.content[0].type === 'text'
        ? message.content[0].text
        : '';

      // Limpiar markdown si existe
      const jsonText = responseText
        .replace(/```json\n?/g, '')
        .replace(/```\n?/g, '')
        .trim();

      const screenConfig = JSON.parse(jsonText);

      return {
```

```
      success: true,
      screen: screenConfig,
      tokensUsed: message.usage.input_tokens + message.usage.output_tokens
    };
  } catch (error) {
    console.error('Error generando pantalla con Claude:', error);
    throw new Error('Failed to generate screen with AI');
  }
 }

 /**
  * Optimiza un prompt para mejorar resultados
  */
 async optimizePrompt(userPrompt: string): Promise<string> {
  const message = await this.client.messages.create({
   model: 'claude-sonnet-4-20250514',
   max_tokens: 500,
   messages: [{
    role: 'user',
    content: `Mejora este prompt para generar una pantalla de digital signage. Hazlo más específico y detallado:

"${userPrompt}"

Responde solo con el prompt mejorado, sin explicaciones adicionales.`
   }]
  });

  return message.content[0].type === 'text' ? message.content[0].text : userPrompt;
 }
}

// Factory para obtener servicio con API key de la organización
export async function getClaudeService(organizationId: string): Promise<ClaudeService> {
 const apiKey = await prisma.apiKey.findUnique({
  where: {
   organizationId_service: {
    organizationId,
    service: 'CLAUDE'
   }
  }
 });

 if (!apiKey) {
  throw new Error('Claude API key not configured');
```

```
  }

  // Desencriptar key (implementar lógica de encriptación)
  const decryptedKey = decrypt(apiKey.key);

  return new ClaudeService(decryptedKey);
}

// Helper para encriptar/desencriptar
function decrypt(encryptedKey: string): string {
  // Implementar con crypto
  // Por ahora, retornar directo (en producción DEBE estar encriptado)
  return encryptedKey;
}
```

3. Ruta API para Generación con IA

typescript

```typescript
// src/api/ai/routes.ts
import { Router } from 'express';
import { authenticate } from '../../middleware/auth';
import { validateRequest } from '../../middleware/validation';
import { z } from 'zod';
import { getClaudeService } from '../../services/ai/claude.service';
import { prisma } from '../../app';

const router = Router();

// Schemas de validación
const generateScreenSchema = z.object({
  prompt: z.string().min(10).max(2000),
  type: z.enum(['signage', 'kiosk', 'dashboard']).optional(),
  orientation: z.enum(['horizontal', 'vertical']).optional()
});

/**
 * POST /api/ai/generate-screen
 * Genera una pantalla con Claude AI
 */
router.post('/generate-screen',
  authenticate,
  validateRequest(generateScreenSchema),
  async (req, res, next) => {
    try {
      const { prompt, type, orientation } = req.body;
      const userId = req.user.id;
      const organizationId = req.user.organizationId;

      // Obtener servicio Claude
      const claudeService = await getClaudeService(organizationId);

      // Generar pantalla
      const result = await claudeService.generateScreen(prompt, type, orientation);

      if (!result.success) {
        return res.status(500).json({
          error: 'Failed to generate screen'
        });
      }

      // Crear la pantalla en la BD
```

```
    const screen = await prisma.screen.create({
      data: {
        name: result.screen.name,
        type: result.screen.type.toUpperCase(),
        zone: 'AI Generated',
        location: 'Virtual',
        orientation: result.screen.orientation.toUpperCase(),
        layout: result.screen.layout,
        content: result.screen,
        aiGenerated: true,
        userId,
        organizationId
      }
    });

    // Actualizar contador de uso de API key
    await prisma.apiKey.update({
      where: {
        organizationId_service: {
          organizationId,
          service: 'CLAUDE'
        }
      },
      data: {
        usageCount: { increment: 1 },
        lastUsed: new Date()
      }
    });

    res.json({
      screen,
      tokensUsed: result.tokensUsed
    });
  } catch (error) {
    next(error);
  }
 }
);

export { router as aiRoutes };
```

⚡ Electron Player - Implementación Completa

1. Main Process

typescript

```typescript
// electron/main/index.ts
import { app, BrowserWindow, ipcMain, screen } from 'electron';
import { autoUpdater } from 'electron-updater';
import path from 'path';
import { WindowManager } from './window';
import { SyncService } from './sync';
import { CacheManager } from './cache';
import { KioskManager } from './kiosk';

/**
 * Main Process - Entry Point del Player
 * Maneja ventanas, actualización, y comunicación IPC
 */

let mainWindow: BrowserWindow | null = null;
let windowManager: WindowManager;
let syncService: SyncService;
let cacheManager: CacheManager;
let kioskManager: KioskManager;

// Player Configuration
const playerConfig = {
  id: process.env.PLAYER_ID || 'PLAYER-' + Math.random().toString(36).substr(2, 9),
  apiUrl: process.env.API_URL || 'http://localhost:3000',
  syncInterval: parseInt(process.env.SYNC_INTERVAL || '30') * 1000, // ms
  kioskMode: process.env.KIOSK_MODE === 'true',
  autoUpdate: process.env.AUTO_UPDATE !== 'false'
};

/**
 * Inicialización de la aplicación
 */
app.whenReady().then(async () => {
  // Inicializar managers
  windowManager = new WindowManager();
  cacheManager = new CacheManager();
  syncService = new SyncService(playerConfig.apiUrl, playerConfig.id);
  kioskManager = new KioskManager();

  // Crear ventana principal
  mainWindow = windowManager.createMainWindow({
    kiosk: playerConfig.kioskMode
  });
```

```
  // Cargar UI
  if (process.env.NODE_ENV === 'development') {
    mainWindow.loadURL('http://localhost:3001');
    mainWindow.webContents.openDevTools();
  } else {
    mainWindow.loadFile(path.join(__dirname, '../renderer/index.html'));
  }

  // Configurar kiosk mode si está habilitado
  if (playerConfig.kioskMode) {
    kioskManager.enable(mainWindow);
  }

  // Iniciar sincronización
  startSync();

  // Configurar auto-updater
  if (playerConfig.autoUpdate) {
    setupAutoUpdater();
  }

  // Registrar IPC handlers
  registerIpcHandlers();

  // Detectar cambios de orientación
  setupOrientationDetection();
});

/**
 * Sincronización con backend
 */
function startSync() {
  // Sync inicial
  syncService.sync().then((data) => {
    if (mainWindow) {
      mainWindow.webContents.send('sync-complete', data);
    }
  });

  // Sync periódico
  setInterval(async () => {
    try {
      const data = await syncService.sync();
```

```
      if (mainWindow) {
        mainWindow.webContents.send('sync-complete', data);
      }
    } catch (error) {
      console.error('Sync error:', error);
      if (mainWindow) {
        mainWindow.webContents.send('sync-error', error);
      }
    }
  }, playerConfig.syncInterval);
}

/**
 * Auto-updater
 */
function setupAutoUpdater() {
  autoUpdater.checkForUpdatesAndNotify();

  autoUpdater.on('update-available', () => {
    if (mainWindow) {
      mainWindow.webContents.send('update-available');
    }
  });

  autoUpdater.on('update-downloaded', () => {
    if (mainWindow) {
      mainWindow.webContents.send('update-ready');
    }
  });

  // Check updates cada hora
  setInterval(() => {
    autoUpdater.checkForUpdates();
  }, 60 * 60 * 1000);
}

/**
 * IPC Handlers
 */
function registerIpcHandlers() {
  // Restart player
  ipcMain.handle('restart-player', async () => {
    app.relaunch();
    app.exit(0);
```

```
  });

  // Clear cache
  ipcMain.handle('clear-cache', async () => {
    await cacheManager.clearAll();
    return { success: true };
  });

  // Take screenshot
  ipcMain.handle('take-screenshot', async () => {
    if (!mainWindow) return { success: false };

    const screenshot = await mainWindow.webContents.capturePage();
    const filename = `screenshot_${Date.now()}.png`;
    const filepath = path.join(app.getPath('userData'), 'screenshots', filename);

    await screenshot.toPNG().then(buffer => {
      require('fs').writeFileSync(filepath, buffer);
    });

    return { success: true, filepath };
  });

  // Get system stats
  ipcMain.handle('get-system-stats', async () => {
    const cpuUsage = process.getCPUUsage();
    const memoryUsage = process.getSystemMemoryInfo();

    return {
      cpu: cpuUsage.percentCPUUsage,
      memory: (memoryUsage.free / memoryUsage.total) * 100,
      uptime: process.uptime()
    };
  });

  // Manual sync
  ipcMain.handle('manual-sync', async () => {
    return await syncService.sync();
  });
}

/**
 * Detección de orientación
 */
```

```
function setupOrientationDetection() {
  screen.on('display-metrics-changed', (event, display, changedMetrics) => {
    if (changedMetrics.includes('rotation')) {
      const primaryDisplay = screen.getPrimaryDisplay();
      const { width, height } = primaryDisplay.size;
      const orientation = width > height ? 'horizontal' : 'vertical';

      if (mainWindow) {
        mainWindow.webContents.send('orientation-changed', orientation);
      }
    }
  });
}

/**
 * Quit handlers
 */
app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit();
  }
});

app.on('activate', () => {
  if (BrowserWindow.getAllWindows().length === 0) {
    // Recrear ventana si no hay ninguna
  }
});
```

2. Window Manager

typescript

```typescript
// electron/main/window.ts
import { BrowserWindow, screen } from 'electron';
import path from 'path';

export class WindowManager {
  /**
   * Crea la ventana principal del player
   */
  createMainWindow(options: { kiosk?: boolean } = {}): BrowserWindow {
    const primaryDisplay = screen.getPrimaryDisplay();
    const { width, height } = primaryDisplay.workAreaSize;

    const window = new BrowserWindow({
      width,
      height,
      fullscreen: options.kiosk,
      kiosk: options.kiosk,
      frame: !options.kiosk,
      show: false, // Mostrar cuando esté listo
      backgroundColor: '#000000',
      webPreferences: {
        preload: path.join(__dirname, '../preload/index.js'),
        nodeIntegration: false,
        contextIsolation: true,
        sandbox: true
      }
    });

    // Mostrar cuando esté listo (evita flash blanco)
    window.once('ready-to-show', () => {
      window.show();
    });

    // Prevenir cerrar accidentalmente en modo kiosk
    if (options.kiosk) {
      window.on('close', (e) => {
        e.preventDefault();
        // Solo permitir cerrar con comando especial
      });
    }

    return window;
```

```
  }
}
```

3. Sync Service

typescript

```typescript
// electron/main/sync.ts
import axios from 'axios';
import { CacheManager } from './cache';

export class SyncService {
  private apiUrl: string;
  private playerId: string;
  private cacheManager: CacheManager;

  constructor(apiUrl: string, playerId: string) {
    this.apiUrl = apiUrl;
    this.playerId = playerId;
    this.cacheManager = new CacheManager();
  }

  /**
   * Sincroniza contenido con el backend
   */
  async sync(): Promise<any> {
    try {
      // Obtener configuración del player
      const response = await axios.get(`${this.apiUrl}/api/player/${this.playerId}/config`, {
        timeout: 10000
      });

      const config = response.data;

      // Guardar en cache
      await this.cacheManager.set('player-config', config);

      // Descargar assets necesarios
      if (config.screen?.content?.video) {
        await this.downloadAsset(config.screen.content.video);
      }

      return config;
    } catch (error) {
      console.error('Sync failed:', error);

      // Usar cache si falla la conexión
      const cachedConfig = await this.cacheManager.get('player-config');
      if (cachedConfig) {
        return cachedConfig;
```

```
    }

    throw error;
  }
 }

 /**
  * Descarga un asset y lo guarda en cache
  */
 private async downloadAsset(url: string): Promise<void> {
  try {
   const response = await axios.get(url, {
    responseType: 'arraybuffer'
   });

   await this.cacheManager.set(`asset-${url}`, response.data);
  } catch (error) {
   console.error('Failed to download asset:', url, error);
  }
 }

 /**
  * Reporta estado del player al backend
  */
 async reportStatus(status: any): Promise<void> {
  try {
   await axios.post(`${this.apiUrl}/api/player/${this.playerId}/status`, status);
  } catch (error) {
   console.error('Failed to report status:', error);
  }
 }
}
```

🚀 Scripts de Deployment

package.json del Player

```
json
```

```
{
 "name": "digital-signage-player",
 "version": "2.0.5",
 "description": "Electron Player para Digital Signage",
 "main": "dist/electron/main/index.js",
 "scripts": {
  "dev": "concurrently \"npm run dev:electron\" \"npm run dev:react\"",
  "dev:electron": "tsc && electron .",
  "dev:react": "vite",
  "build": "tsc && vite build",
  "package:win": "electron-builder --win",
  "package:mac": "electron-builder --mac",
  "package:linux": "electron-builder --linux",
  "package:all": "electron-builder -mwl",
  "publish": "electron-builder --publish always"
 },
 "build": {
  "appId": "com.digitalsignage.player",
  "productName": "Digital Signage Player",
  "directories": {
   "output": "release"
  },
  "files": [
   "dist/**/*",
   "package.json"
  ],
  "win": {
   "target": [
    {
     "target": "nsis",
     "arch": ["x64", "ia32"]
    }
   ],
   "icon": "build/icon.ico"
  },
  "mac": {
   "target": ["dmg", "zip"],
   "icon": "build/icon.icns",
   "category": "public.app-category.business"
  },
  "linux": {
   "target": ["AppImage", "deb"],
   "icon": "build/icon.png",
```

```
    "category": "Utility"
  },
  "publish": {
    "provider": "github",
    "owner": "tu-usuario",
    "repo": "digital-signage-player"
  }
 }
}
```

📝 Documentación de Uso

Para Desarrolladores

1. **Clonar repositorio**

```bash
git clone https://github.com/tu-usuario/digital-signage.git
cd digital-signage
```

2. **Instalar dependencias**

```bash
npm install
```

3. **Configurar variables de entorno**

```bash
cp .env.example .env
# Editar .env con tus configuraciones
```

4. **Iniciar base de datos**

```bash
docker-compose up -d postgres redis minio
```

5. **Ejecutar migraciones**

```bash
npx prisma migrate dev
```

6. **Iniciar en desarrollo**

```bash
# Backend
cd packages/backend
npm run dev

# Admin Panel
cd packages/admin
npm run dev

# Electron Player
cd packages/player
npm run dev
```

Para Producción

1. **Build de aplicaciones**

```bash
npm run build:all
```

2. **Deploy con Docker**

```bash
docker-compose -f docker/docker-compose.prod.yml up -d
```

3. **Crear instaladores Electron**

```bash
cd packages/player
npm run package:all
```

🔒 Seguridad

Mejores Prácticas Implementadas

1. **Encriptación de API Keys**: Todas las API keys se almacenan encriptadas
2. **JWT con refresh tokens**: Sistema de autenticación robusto
3. **Rate limiting**: Protección contra abuso de API
4. **CORS configurado**: Solo dominios permitidos
5. **Helmet.js**: Headers de seguridad HTTP
6. **Input validation**: Validación con Zod en todos los endpoints
7. **SQL Injection protection**: Prisma ORM previene inyecciones
8. **XSS protection**: Content Security Policy configurado

📊 Monitoreo y Logs

Configuración de Grafana

yaml

```
# docker/grafana/dashboards/player-dashboard.json
{
  "title": "Digital Signage Players",
  "panels": [
    {
      "title": "Players Online",
      "type": "stat",
      "targets": [
        {
          "query": "sum(player_online)"
        }
      ]
    },
    {
      "title": "Sync Status",
      "type": "timeseries",
      "targets": [
        {
          "query": "rate(player_sync_total[5m])"
        }
      ]
    }
  ]
}
```

Este es un sistema enterprise-grade completo. ¿Quieres que profundice en alguna parte específica o que agregue más funcionalidades?